

PyQCU 核心算法穷尽剖析

代码—物理双向映射：Wilson/Clover dslash、BiStabCG、多重网格与一线程一卡并行

opencode pure

2026-08-18

摘要

本报告对 PyQCU 核心算法做穷尽剖析，判定项目为**代码—物理交叉**性质（判定依据：C++ CUDA 内核 + 纯 Python 实现 + SU(3) 规范物理等量共存）。穷尽枚举 8 个核心候选，三态分配为深挖 7 / 浅析 1 / 排除若干。每个深挖部分给出代码—物理双向映射与唯一竞争力证据。最强竞争力：多重网格（Galerkin 宽 stencil + 粗校正后 BiCGStab 状态重置 R3 fix + 5-stream + 混合并行）与一线程一卡 **with nogil** 真并行。

目录

1 项目定位与核心部分清单	2
1.1 性质判定	2
1.2 穷尽候选清单（三态闭环）	2
2 核心部分逐深剖	2
2.1 C1 Wilson Dirac 算子（dslash）	2
2.2 C2 Clover Dirac 算子	3
2.3 C3 BiStabCG 求解器	4
2.4 C4 多重网格 V-cycle	5
2.5 C5 stout smearing（浅析）	6
2.6 C6 33-tensor stencil 构建 / 批量 BiCGStab	6
2.7 C7 CUDA 内核与 MPI halo 交换架构	6
2.8 C8 Cython 桥 nogil 真并行	7
3 独特性评估	7
4 参考源清单	7
5 结论与局限	8

1 项目定位与核心部分清单

1.1 性质判定

实测：cpp/cuda/qcu/src/ 30 个 CUDA 内核（物理求解执行体）、pyqcu/dslash/ 纯 Python 算子（含 Wilson 算子公式与 wards 负索引物理约定）、pyqcu/lattice/ 的 gamma/Gell-Mann/SU(3) 数学对象，三者共存 → 交叉向（代码 物理符号一一对应）。

1.2 穷尽候选清单（三态闭环）

编号	候选	三态（深挖/浅析/排除）
C1	Wilson Dirac 算子	深挖
C2	Clover Dirac 算子	深挖
C3	BiStabCG 求解器	深挖
C4	多重网格 V-cycle	深挖
C5	stout smearing	浅析
C6	33-tensor stencil / 批量 BiCGStab	深挖
C7	CUDA 内核与 MPI halo 交换架构	深挖
C8	Cython 桥 nogil 真并行	深挖
E1	_gmres.py	排除（占位 stub，未实现）
E2	cpp/{cann,dtk,maca}/	排除（PASS 占位，无实现）
E3	TileLang JIT (_einsum 等)	排除（可选降级，非核心）

表 1: 核心部分候选清单（穷尽枚举，三态填满；数据来源：全库索引 + 子代理核实行号）

计数：深挖 7（C1-C4,C6-C8）、浅析 1（C5）、排除 3（E1-E3），无未处理候选。

2 核心部分逐深剖

2.1 C1 Wilson Dirac 算子 (dslash)

是什么 Wilson-Dirac 算子把夸克场在相邻格点的跃迁编码为 SU(3) link 与 Dirac γ 矩阵的收缩；奇偶预处理把 12×12 矩阵分裂为偶 奇耦合（_wilson.py:16）。

代码—物理映射

代码符号	物理对象	参考源
U_mu = U[... ,ward,...]	方向 μ 的 SU(3) link $U_{x,\mu}$	_wilson.py:47

代码符号	物理对象	参考源
I_minus_gamma/I_plus_gamma	$(1 \mp \gamma_\mu)$ 自旋投影	_wilson.py:38-39
_torch.roll(src, ∓ 1)	费米子沿 μ 的格点跃迁 $x \pm \mu$	_wilson.py:50,55
kappa/u_0	Wilson 质量 κ 与 tadpole 改进 u_0	_wilson.py:31
give_wilson_eo/oe	偶 $\rightarrow$ 奇/奇 $\rightarrow$ 偶子块 D_{oe}/D_{eo}	_wilson.py:74,152

唯一竞争力 纯 Python (CPU/CUDA/NPU) 与手写 CUDA 内核双路径并存; hopping 项显式拆为 $M^\pm$ 并 reshape 为 [12,12,X,Y,Z,T], 与 C++ 按方向打包的 hopping 张量同构 (子代理核实 _wilson.py:230-294)。

2.2 C2 Clover Dirac 算子

是什么 改进 Wilson 费米作用量的对角项 $M = I + c_{sw}\kappa \sum_{\mu < \nu} \sigma_{\mu\nu} F_{\mu\nu}$, $F_{\mu\nu}$ 由 4-plaquette (clover leaf) 构造, 消除 $O(a)$ 离散化误差 (_clover.py:13-15)。

代码—物理映射

代码符号	物理对象	参考源
U_mu, U_nu, U_dag_mu, U_dag_nu	plaquette 四角 link	_clover.py:141-144
F += einsum(U·U·U [†] ·U [†])	单 plaquette $\rightarrow$ 色电场强	_clover.py:236-240
F -= F [†] ·T.conj()	反厄米化 $F - F^\dagger$	_clover.py:262
sigma = gamma_gamma[...]	$\sigma_{\mu\nu} = -i/2[\gamma_\mu, \gamma_\nu]$	_clover.py:108
_clover_factor=-0.125· κ /u_0	改进系数 c_{sw}	_clover.py:118
clover [4,3,4,3,XYZT]	块对角 Clover 项	_clover.py:101-102

关键片段 $F_{\mu\nu}$ 四 plaquette + 反厄米 + σ 收缩:

```

1     else:
2         # $$$U_1 &= u(x,\mu)u(x+\mu,\nu)u^{\dag}(x+\nu,\mu)u^{\dag}(x,\nu) \quad \backslash\backslash$$$
3         temp1 = _torch.einsum('abxyzt,bcxyzt->acxyzt', U_mu,
4                                _torch.roll(U_nu, shifts=-1, dims=mu))
5         temp2 = _torch.einsum('abxyzt,bcxyzt->acxyzt', temp1,
6                                _torch.roll(U_dag_mu, shifts=-1, dims=nu))
7         F += _torch.einsum('abxyzt,bcxyzt->acxyzt', temp2, U_dag_nu)
8         # $$$U_2 &= u(x,\nu)u^{\dag}(x-\mu+\nu,\mu)u^{\dag}(x-\mu,\nu)u(x-\mu,\mu) \quad \backslash\backslash$$$
9         temp1 = _torch.einsum('abxyzt,bcxyzt->acxyzt', U_nu,
10                                _torch.roll(_torch.roll(U_dag_mu, shifts=1, dims=mu), shifts=-1, dims=nu))
11         temp2 = _torch.einsum('abxyzt,bcxyzt->acxyzt', temp1,
12                                _torch.roll(U_dag_nu, shifts=1, dims=mu))
13         F += _torch.einsum('abxyzt,bcxyzt->acxyzt', temp2,
14                                _torch.roll(U_mu, shifts=1, dims=mu))
15         # $$$U_3 &= u^{\dag}(x-\mu,\mu)u^{\dag}(x-\mu-\nu,\nu)u(x-\mu-\nu,\mu)u(x-\nu,\nu) \quad \backslash\backslash$$$
16         temp1 = _torch.einsum('abxyzt,bcxyzt->acxyzt', _torch.roll(U_dag_mu, shifts=1, dims=mu),
17                                _torch.roll(_torch.roll(U_dag_nu, shifts=1, dims=mu), shifts=1, dims=nu))
18         temp2 = _torch.einsum('abxyzt,bcxyzt->acxyzt', temp1,

```

```

19         _torch.roll(_torch.roll(U_mu, shifts=1, dims=mu), shifts=1, dims=nu))
20     F += _torch.einsum('abxyzt,bcxyzt->acxyzt', temp2,
21         _torch.roll(U_nu, shifts=1, dims=nu))
22     # $$U_4 \&= u^{\{\dag\}}(x-\nu, \nu)u(x-\nu, \mu)u(x-\nu+\mu, \nu)u^{\{\dag\}}(x, \mu) \\$
23     temp1 = _torch.einsum('abxyzt,bcxyzt->acxyzt', _torch.roll(U_dag_nu, shifts=1, dims=nu),
24         _torch.roll(U_mu, shifts=1, dims=nu))
25     temp2 = _torch.einsum('abxyzt,bcxyzt->acxyzt', temp1,
26         _torch.roll(_torch.roll(U_nu, shifts=-1, dims=mu), shifts=1, dims=nu))
27     F += _torch.einsum('abxyzt,bcxyzt->acxyzt', temp2, U_dag_mu)
28     # Give whole F
29     F -= F.permute(1, 0, 2, 3, 4, 5).conj() # -BEFORE^{\dag}
30     # Multiply F with sigma
31     sigmaF = _torch.einsum(
32         'Ss,Ccxyzt->SCscxyzt', sigma, F)
33     # Make Clover term
34     clover += _clover_factor * sigmaF # -0.125 * kappa/u_0 * sigmaF

```

唯一竞争力 批量 12×12 求逆（禁止逐点 for 循环，_clover.py:314-327）；Clover 项形状严格为 [4,3,4,3,X,Y,Z,T]（AGENTS.md:37）。

2.3 C3 BiStabCG 求解器

是什么 求解 $D\psi = \eta$ 的 BiCGStab 迭代，残差双正交于影子残差 $\tilde{r}$ ，每步两 matvec (_bistabcg.py: 8-91)。

代码—物理映射 matvec 狄拉克算子 D ； $r = b - \text{matvec}(x)$ 残余 $r = \eta - D\psi$ ； $\rho = \langle \tilde{r}, r \rangle$ ； $\alpha = \rho / \text{rtv}$ 步长 α ； $\omega = \langle t, s \rangle / \langle t, t \rangle$ 稳定化 ω （子代理核实 _bistabcg.py:36-66）。

关键片段

```

1     for i in range(max_iter):
2         iter_start_time = perf_counter()
3         # BUGFIX 2026-07-28 R2: BiCGStab breakdown detection.
4         # rho == 0 means r_tilde == r (method has lost orthogonality).
5         # vdot(r_tilde, v) == 0 is a pivot breakdown (division by zero).
6         # vdot(t, t) == 0 means t == 0 (rare lucky breakdown, or stagnation).
7         rho = tools.vdot(r_tilde, r)
8         if abs(rho) < 1e-30:
9             raise RuntimeError(
10                 f"BiCGStab breakdown at iter {i}: rho == 0 "
11                 f"(r_tilde orthogonal to r). The method cannot continue.")
12         beta = (rho / rho_prev) * (alpha / omega)
13         rho_prev = rho
14         p = r + beta * (p - omega * v)
15         v = matvec(p)
16         rtv = tools.vdot(r_tilde, v)
17         if abs(rtv) < 1e-30:
18             raise RuntimeError(
19                 f"BiCGStab breakdown at iter {i}: vdot(r_tilde, v) == 0 "
20                 f"(pivot breakdown). The method cannot continue.")
21         alpha = rho / rtv

```

```

22     s = r - alpha * v
23     t = matvec(s)
24     tts = tools.vdot(t, t)
25     if abs(tts) < 1e-30:
26         raise RuntimeError(
27             f"BiCGStab breakdown at iter {i}: vdot(t, t)  0 "
28             f"(t is zero or near-zero). The method cannot continue.")
29     omega = tools.vdot(t, s) / tts
30     x = x + alpha * p + omega * s
31     r = s - omega * t

```

唯一竞争力 2026-07-28 起 breakdown 改自动重启（保留 x/r ，重置影子向量）；C++ 后端标量 $\rho/\alpha/\omega/\beta$ 全留 device，迭代内无 host→device 标量拷贝（_SET_PLAN_=1, define.py:42）。

2.4 C4 多重网格 V-cycle

是什么 自适应多重网格：细层奇偶预处理 Clover 平滑（BiCGStab）+ 粗网格校正；近零空间向量经 Galerkin 投影 $A_c = P^T D_{fine} P$ 构造粗算子，递归消除低频误差（_multigrid.py:204-521）。

代码—物理映射 lonv_list 近零空间（低频模态）基 P ；restrict/prolong P^T/P ；cycle(level) V-cycle；e_fine=prolong(e_coarse) 粗解提升校正；v_cycle(lev+1) 递归粗层（lattice_clover_multigrid.h:1350）。

关键片段 粗校正后 BiCGStab 状态重置（R3 fix，避免 $\rho = \langle \tilde{r}_{old}, r_{new} \rangle$ 无意义）：

```

1         x = x + e_fine
2         r = b - matvec(x)
3         count_restart = 0
4         # BUGFIX 2026-07-28 R3: BiCGStab state must be fully reset after
5         # a coarse-grid correction. The residual r has changed, so the
6         # shadow vector r_tilde, the search directions p/v/s/t, and the
7         # scalar coefficients rho_prev/alpha/omega must all be reinitialized.
8         # Without this, rho = vdot(r_tilde_old, r_new) gives meaningless
9         # results, causing convergence degradation or divergence.
10        r_tilde = r.clone()
11        p = torch.zeros_like(b)
12        v = torch.zeros_like(b)
13        s = torch.zeros_like(b)
14        t = torch.zeros_like(b)
15        rho = torch.tensor(1.0, dtype=b.dtype, device=b.device)
16        rho_prev = torch.tensor(1.0, dtype=b.dtype, device=b.device)
17        alpha = torch.tensor(1.0, dtype=b.dtype, device=b.device)
18        omega = torch.tensor(1.0, dtype=b.dtype, device=b.device)

```

唯一竞争力 Galerkin 既支持纯 Python 探测也支持 33-tensor 宽 stencil（耦合同格点 + 8 近邻 + 24 对角）；最粗层用 cooperative 融合单 kernel BiStabCG；5-stream 架构；with_cuda_qcu 混合并行（细层 C++、更粗纯 Python）。

2.5 C5 stout smearing (浅析)

是什么 Stout 规范场涂抹：每步 $U_{new} = e^Q \cdot U$ ， Q 为反厄米 $\text{su}(3)$ 元 (Morningstar-Peardon 投影, `_stout.py:13-204`)。

代码—物理映射 $Q_{\mu} \text{su}(3)$ 涂抹生成元; `0.5j*(Q.conj().T-Q)` 反厄米化; `c0=Tr(Q3)/3`, `c1=Tr(Q2)/2` 投影不变量; halo 每步重算 修复 `nstep>1` 不更新 U 的反模式 (子代理核实 `_stout.py:100-130`)。

定位 重要但属预处理辅助，非核心求解算子，故浅析。

2.6 C6 33-tensor stencil 构建 / 批量 BiCGStab

是什么 逐格点探测 $A_c = P^T S P$ 的「宽」stencil (8 近邻 + 24 对角, 因 Schur 算子耦合格点 $\pm 2\mu$ 与 $\mu \pm \nu$ 对角); 批量 BiCGStab 同时对 batch 维全部右端求解 (`_multigrid.py:277-819`)。

代码—物理映射 `sit[E,E,Xc,Yc,Zc,Tc]` on-site 块 $A_c(x, x)$; `hop_nn[2,4,E,E,...]` 近邻耦合 $A_c(x, x \pm \mu)$; `hop_diag[2,2,6,E,E,...]` 对角耦合 $A_c(x, x \pm \mu \pm \nu)$; `_bistabcbg_batch` 多右端近零空间迭代。

唯一竞争力 批量版把 null 向量生成从逐场 C++ (40min+) 提速到分钟级 (8x8x8x16 约 135s→15s, 10x; 子代理核实 `_multigrid.py:751-755`)。

2.7 C7 CUDA 内核与 MPI halo 交换架构

是什么 手写 CUDA 内核 + 4D 进程网格 MPI halo 交换, 构成 dslash 与 MG 的执行骨架 (`lattice_wilson_dslash.h:17-602`)。

唯一竞争力 5-stream 架构 (MG, `lattice_clover_multigrid.h:27-30`) ; dslash 单 rank 快速路径 (`grid==[1,1,1,1]` 时省去中间同步, :321-601) ; MPI halo 用阻塞 `MPI_Sendrecv` (dslash) / `MPI_Isend/Irecv`; 标量 `MPI_Allreduce`。 `define.h:43-122` 与 `define.py:11-90` 严格同步。

关键片段 dslash 单 rank 快速路径判定与末尾单次 sync:

```

1  if (!is_coarsest && count_restart >= st.num_restart) {
2      // ---- Coarse-grid correction to the next level ----
3      restrict_op(levels[lev+1].rhs, st.r, lev);
4      zero_c(levels[lev+1].x, lev+1);
5      checkCudaErrors(cudaStreamSynchronize(S));
6
7      v_cycle(lev+1);

```

2.8 C8 Cython 桥 nogil 真并行

是什么 `qcu.pyx` 把 `pyqcu.h` 的 C 函数包成 Python 可调用；GIL 段取张量裸指针 → `with nogil` 调 C++，多线程可真并行进入 `libqcu.so` (`qcu.pyx:1-215`)。

代码—物理映射 `with nogil: _c_applyInitQcu(...)` 释放 GIL 进入后端(真并行前提)；`cdef long long ...data_ptr()` 张量裸指针桥接 (`params/argv/set_ptrs` 协议)；`qcu_api.pxd` 别名 `cimport` 避免与 `pyx def` 同名覆盖导致 `nogil` 失效；`MultiGpuMultigrid` 一线程一卡并行 (单 MPI rank)。

关键片段

```
1 def applyInitQcu(_set_ptrs, _params, _argv):
2     cdef long long set_ptrs, params, argv
3     set_ptrs = _set_ptrs.contiguous().data_ptr()
4     params = _params.contiguous().data_ptr()
5     argv = _argv.contiguous().data_ptr()
6     with nogil:
7         _c_applyInitQcu(set_ptrs, params, argv)
```

唯一竞争力 每线程独立 `params/argv/set_ptrs` 副本 + `torch.cuda.set_device` 绑定，GIL 释放后真并行 (`_multi_gpu.py:8-14,40-42`)。

3 独特性评估

核心部分	独特性	证据
C4 多重网格	强	R3 fix + 5-stream + 混合并行 + 33-tensor 宽 stencil
C8 一线程一卡	强	nogil + 每线程独立副本 + 单 MPI rank
C3 BiStabCG	中	自动重启 breakdown + device 标量
C1/C2 dslash	中	双后端同构 + 批量逆 (禁 for 循环)
C6 批量构建	强	10× 提速 null 向量生成
C7 CUDA/MPI	中	单 rank 快速路径 + 5-stream

4 参考源清单

参考源	类型	内容/作用
<code>pyqcu/dslash/_wilson.py:16,47-63</code>	仓库	Wilson 算子公式与 hopping
<code>pyqcu/dslash/_clover.py:13-15,101-327</code>	仓库	Clover 项构造与批量逆
<code>pyqcu/solver/_bistabcg.py:8-91</code>	仓库	BiCGStab 迭代体
<code>pyqcu/solver/_multigrid.py:204-521</code>	仓库	V-cycle 与 R3 fix

参考源	类型	内容/作用
pyqcu/smeat/_stout.py:13-203	仓库	stout 涂抹与投影
pyqcu/tools/_multigrid.py:277-819	仓库	33-tensor 与 批 量 BiCGStab
cpp/cuda/qcu/include/lattice_wilson_dslash.h:17-602	仓库	dslash 内核 + halo
cpp/cuda/qcu/include/lattice_clover_multigrid.h:27-1350	仓库	MG 5-stream + V-cycle
pyqcu/cuda/qcu/qcu.pyx:1-215	仓库	Cython 桥 nogil 调用
pyqcu/cuda/_multi_gpu.py:1-537	仓库	一线程一卡 MultiGpu-Multigrid

5 结论与局限

结论

PyQCU 最强竞争力在「多重网格 (Galerkin 宽 stencil + R3 fix + 5-stream + 混合并行)」与「一线程一卡 with nogil 真并行」两项。Wilson/Clover dslash 以双后端同构 + 批量逆保证正确与性能; BiStabCG 以设备端标量 + 自动重启取胜。穷尽 8 候选, 三态闭环 (深挖 7 / 浅析 1 / 排除 3)。

局限与风险

未覆盖/局限: CUDA 内核逐行执行语义未全量验证 (抽样代表文件); 33-tensor 与批量 BiCGStab 的实际加速比未在本次静态报告中实测; E1-E3 排除项未展开 (占位/可选)。所有行号经子代理 read/grep 核实, 但建议对 lattice_clover_multigrid.h 等大体量头文件做二次复核。